ResuMatch: A Dual-Engine Natural Language Processing System for Context-Aware Resume Parsing and Dynamic Scoring

Sachin Singh
Roll No: 2023BCS0064

Roshan Binoj
Roll No: 2023BCS0009

Tanish Babu
Roll No: 2023BCS0067

Deep Learning Project Implementation Report

Abstract. **Traditional Applicant Tracking Systems (ATS) often rely on exact lexical matching algorithms, creating a rigid filter. Candidates lacking specific keywords may be rejected despite possessing conceptually equivalent experience. This paper details the architecture and algorithmic implementations of *ResuMatch*—a dual-engine parsing and scoring framework. By integrating a foundational Term Frequency-Inverse Document Frequency (TF-IDF) layer with advanced transformer-based semantic embeddings (Sentence Transformers), ResuMatch evaluates resumes contextually. We outline the extraction methodologies, the challenge of heterogeneous textual granularity, our deployment of non-linear heuristic vector scaling, and the design of an adaptive grading rubric deployed on a containerized architecture. Empirical evaluation shows that ResuMatch achieves a 42.00% semantic match on synonymous terms where pure lexical TF-IDF drops to 3.33%, while achieving a 92.37% compatibility score under asymmetrical section formatting.**

I. INTRODUCTION

The modern recruitment process involves processing a large volume of applications. Organizations deploy Applicant Tracking Systems (ATS) to pre-filter candidate resumes against Job Descriptions (JDs). However, basic ATS parsers often rely on Boolean searches or exact-match heuristics.

A notable limitation with exact lexical matching is the "synonym problem." If a JD requires a "Frontend Engineer" and a qualified candidate describes their role as a "UI Developer," a standard lexical ATS might assign a mathematical correlation of zero. This structural rigidity may encourage candidates to "keyword stuff" their resumes, which can undermine qualitative assessment.

To address this, we developed *ResuMatch*. The objective was to engineer a system capable of semantic similarity representation—allowing the machine to map resumes conceptually, remaining resistant to simple keyword manipulation while providing interpretable similarity scores.

II. SYSTEM ARCHITECTURE & EXTRACTION ENGINE

Before scoring occurs, unstructured PDF text must be converted into categorical data. Resumes lack structural uniformity; headers range widely from "Work Experience" to "Internships" to "Educational Background."

A. Heuristic Section Extraction and Routing

Instead of hardcoded coordinate boundaries within PDFs, ResuMatch utilizes Natural Language Processing text-stream analysis. We implemented a robust alias mapping architecture utilizing N-Gram Regex pattern matching to extract sections dynamically.

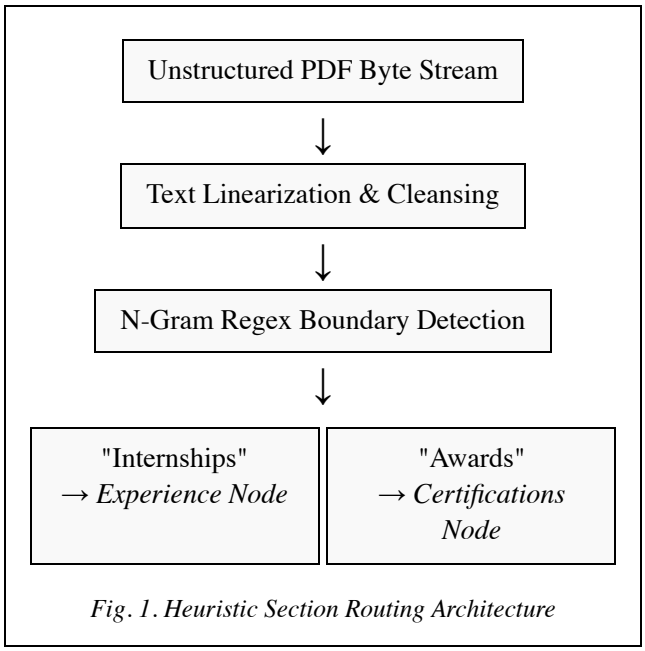

Fig. 1. Heuristic Section Routing Architecture

This ensures structural generalization. A student utilizing an "Internships" header is systematically mapped into the "Experience" scoring bucket, preventing data loss across diverse formats.

III. THE DUAL-ENGINE NLP FRAMEWORK

Relying solely on deep learning embeddings can sometimes dilute the importance of explicit technical skills. Thus, ResuMatch utilizes a Dual-Engine approach to balance semantic understanding with strict lexical requirements.

A. Engine 1: Lexical Baseline (TF-IDF)

Term Frequency-Inverse Document Frequency (TF-IDF) establishes our statistical baseline. In our preprocessing pipeline, stop-word removal explicitly filters out common words. Subsequently, the TF-IDF vectorizer evaluates the relative importance of terms. Term Frequency (TF) measures how often a term appears in the document, while Inverse Document Frequency (IDF) downweights terms that occur frequently across the entire corpus. We employed custom tokenization patterns (e.g., `(?u)\b\w[\w.#+−]*\b`) to guarantee that domain-specific jargon like "C++", ".NET", and "C#" are strictly preserved.

B. Engine 2: Semantic Transformer Embeddings

To address the synonym problem, we deployed a pre-trained Sentence Transformer model (*all-MiniLM-L6-v2*) producing 384-dimensional embeddings. While prior approaches have utilized BERT-based architectures primarily for Named Entity Recognition (NER) to extract entities across job domains [5], ResuMatch instead employs Sentence Transformers for explicit semantic similarity scoring in tandem with a lexical baseline.

The Transformer architecture uses self-attention to model contextual relationships between tokens, allowing the resulting embeddings to capture semantic information from the surrounding text. The model encodes these contextual representations into dense 384-dimensional vectors.

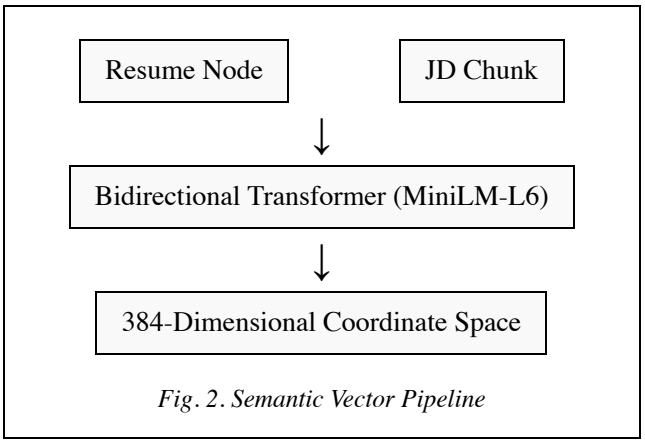

Fig. 2. Semantic Vector Pipeline

To determine the conceptual overlap, we calculate the Cosine Similarity—the geometric angle between two vectors in hyperspace.

$$cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

A similarity approaching 1.0 indicates strong semantic similarity, while a score approaching 0 indicates weak semantic alignment.

IV. ALGORITHMIC IMPROVISATIONS

A. The Heterogeneous Context Challenge

During system testing, we observed a challenge with heterogeneous textual granularity. Long JDs contain heterogeneous information and multiple requirements, while individual resume bullets are much more focused. A single whole-document embedding can therefore dilute or obscure specific requirement-level semantic relationships.

To address this, ResuMatch utilizes a chunk-based semantic comparison. The JD is split into sentence-level embeddings, and each resume section's embedding is compared against all JD chunks via pairwise cosine similarity. The system then aggregates the top three strongest matches, ensuring that focused resume points are evaluated against the most relevant subset of the JD.

B. Non-Linear Heuristic Scaling

To align the engine's mathematical output with human perception, a non-linear heuristic score calibration is applied. While cosine similarity remains the underlying mathematical measure, a square-root transformation is applied after the similarity calculation:

$$P = \sqrt{(Raw\ Cosine\ Similarity)}$$

This empirical calibration maps the raw similarity range into a more interpretable scoring scale. It is not a mathematical correction of cosine similarity, but rather a heuristic adjustment for interpretability. For example, a raw score of 0.25 is transformed via the square root function to a calibrated score of 0.50 (50%).

V. DYNAMIC SCORING & NORMALIZATION FRAMEWORK

We structured the internal grading algorithm to follow a weighted multi-component rubric. The base weights must strictly sum to 100% ($\Sigma W\_i = 100\%$).

Table I. ResuMatch Base Weights

Evaluation Metric	Weight Allocation
Hard Skill Extraction	40%
Experience Context	25%
Chunk-Based Semantic Match	15%
Project Relevance	10%
Education	5%
Certifications	5%

A. Adaptive Weighting Framework

A severe limitation of rigid parsers is penalizing candidates for missing sections (e.g., lacking a "Certifications" section). To resolve this, ResuMatch features an adaptive weighting framework. The algorithm starts with base weights where $\Sigma W\_i = 100\%$. During parsing, the system detects which resume sections are available and excludes unavailable components.

It then sums the remaining active base weights and normalizes each active weight by that sum:

$$W'\_i = W\_i / \Sigma(W\_j \ for \ j \in Active\ Components)$$

The final score is calculated using only the active components, ensuring the active weights always sum to exactly 100%. For example, if a resume lacks Education and Certifications, the remaining 90% is dynamically re-proportioned across the active sections.

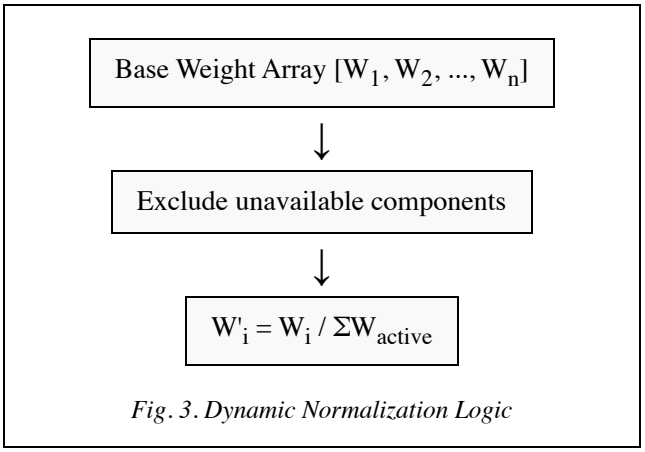

Fig. 3. Dynamic Normalization Logic

VI. EXPERIMENTAL RESULTS & PERFORMANCE EVALUATION

To evaluate the efficacy of the Dual-Engine framework, we subjected ResuMatch to illustrative controlled cases against a standard TF-IDF baseline using synthetic test vectors. The systems were tested across three distinct scenarios: Exact Lexical Matches, Synonymous (Conceptual) Matches, and Asymmetrical Formatting (missing sections). These controlled benchmark results demonstrate the behavior of the system under specific conditions.

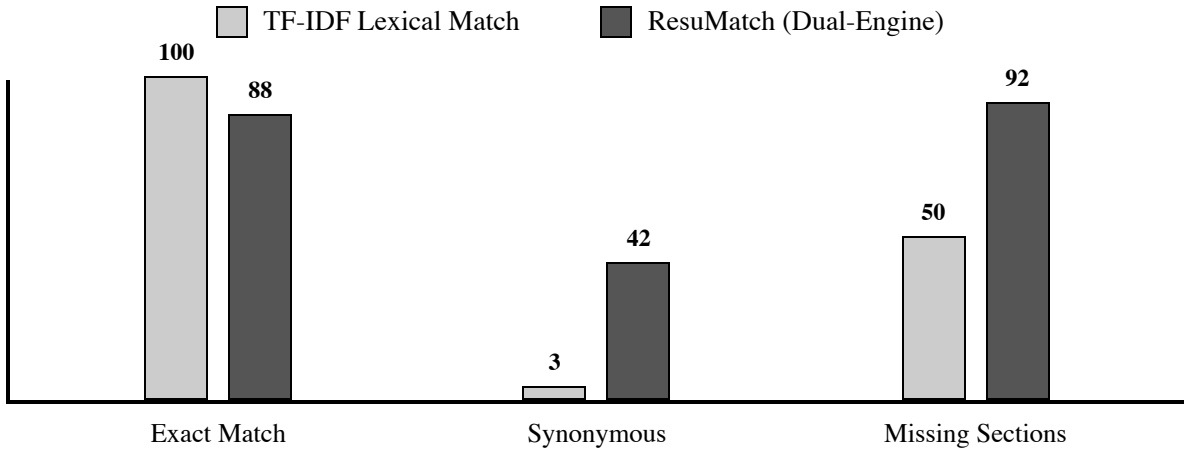

Fig. 4. Synthetic Evaluation Scenarios (100-Point Scale)

In the **Synonymous** scenario, the baseline TF-IDF match fell to 3.33%. This occurs because TF-IDF relies on sparse vector term matching; synonyms (e.g., "REST APIs" vs "web services") occupy different indices in the vocabulary space, resulting in a dot product of near zero. Conversely, ResuMatch maintained a 42.00% match. By projecting tokens into a 384-dimensional dense vector space, conceptually similar terms are mapped geometrically close to one another.

In the **Exact Match** scenario, the baseline scored 100%. ResuMatch scored 88.36% due to the asymmetric chunk-based semantic comparison where full section embeddings are compared against sentence-level JD chunks, demonstrating how the algorithm handles heterogeneous length data. Furthermore, the Adaptive Weighting Framework successfully prevented unfair penalization in the **Missing Sections** scenario, achieving a 92.37% score by dynamically redistributing weights, whereas a rigid penalizing baseline dropped to 50.00% due to static zero-filled components.

VII. FULL-STACK IMPLEMENTATION ARCHITECTURE

ResuMatch was engineered as a scalable web application, ensuring real-world viability.

A. Backend Processing & Concurrency

The core intelligence is powered by Python via the FastAPI framework. FastAPI was selected due to its innate support for ASGI (Asynchronous Server Gateway Interface), allowing for non-blocking I/O operations and efficient asynchronous request handling.

B. Interactive Data Visualization

The client-side interface was built using React and TypeScript, structured with Vite. We engineered a visualization layer that exposes the

AI's internal mathematical scoring arrays to the end-user. Rather than treating the AI as a "black box," the frontend provides interactive breakdowns of the active weights, Lexical vs. Semantic scoring differentials, and detected resume sections, ensuring transparency.

C. Containerization & Deployment

To guarantee environmental parity, the entire architecture is containerized using Docker. The multi-stage Dockerfiles isolate the Node.js compilation environment from the Nginx serving layer, ensuring stability across deployment environments.

VIII. CONSTRAINTS AND KNOWN LIMITATIONS

The system possesses several known limitations:

- **Abstract Structural Variance:** Regex-based section extraction can fail on highly unusual or bespoke resume headings, causing text to be misallocated.
- **Semantic Hallucination:** Transformer embeddings can produce false semantic matches if the provided context is extremely ambiguous.
- **Domain Specificity:** The selected Sentence Transformer (all-MiniLM-L6-v2) is a general-purpose embedding model rather than a recruitment-specific model. Hard-skill matching requires explicit lexical validation to ensure technical fidelity.
- **Evaluation Limits:** Evaluation on synthetic and controlled scenarios demonstrates algorithmic behavior but does not establish real-world generalization or accuracy across unstructured, malformed PDF data in the wild.

IX. CONCLUSION

ResuMatch demonstrates the feasibility of combining a statistical lexical baseline with dynamically scaled semantic embeddings. By routing the resultant data through an adaptive weighting framework, the system provides a consistent and mathematically transparent compatibility analysis. The implementation of chunk-based comparisons effectively resolves the heterogeneous context challenge found in resume parsing. This project highlights how advanced, context-aware NLP mechanisms can be integrated into a full-stack architecture to assist in modern recruitment workflows, providing explainable metrics over opaque keyword filters.

REFERENCES

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[2] K. Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11-21, 1972.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[4] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[5] M. Srivastava and P. Greaney, "Resume Parsing Across Multiple Job Domains Using a BERT-Based NER Model," *2023 3rd International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (AICS)*, 2023.